

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-АВТОТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ ALIEXPRESS

Команда

Степанов, Шакуров,
Кислица, Миридаштаки

Руководитель

А.М. Шевелёва

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: С.А. Кислица

Группа: 4388

Тема практики: UI-тестирование веб-приложения AliExpress

Задание на практику:

Разработать набор UI-автотестов для проверки основных пользовательских сценариев веб-приложения AliExpress.

В ходе работы необходимо составить чек-лист из различных сценариев (поиск, фильтрация, навигация и т.д.), спроектировать структуру тестового проекта на основе паттерна Page Object Model, реализовать на Java с использованием Selenide, JUnit 5 и AssertJ все запланированные тесты с применением динамических ожиданий, провести запуск тестов, зафиксировать результаты и оформить отчёт по практике.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент

С.А. Кислица

Руководитель

А.М. Шевелёва

АННОТАЦИЯ

Отчёт содержит результаты учебной практики по автоматизации UI-тестирования интернет-магазина AliExpress. В ходе работы разработан чек-лист, включающий 15 ключевых пользовательских сценариев: от поиска товаров до работы с корзиной и дополнительными элементами интерфейса. Спроектирована архитектура тестового проекта на основе паттерна Page Object Model, реализованы классы элементов, страниц и тестов. Автоматизированы проверки, выполненные на Java 17 с использованием фреймворков Maven, JUnit 5, Selenide и AssertJ. В отчёте описаны структура проекта, подходы к реализации, возникавшие сложности (включая защиту от ботов) и итоговые результаты тестирования.

SUMMARY

The report presents the results of the educational practice on UI test automation for the AliExpress online store. During the work, a checklist of 15 key user scenarios was developed, covering from product search to cart operations and additional interface elements. The test project architecture was designed following the Page Object Model pattern, with implemented classes for elements, pages, and tests. The automated checks were implemented in Java 17 using the Maven, JUnit 5, Selenide and AssertJ frameworks. The report describes the project structure, implementation approaches, challenges encountered (including bot protection mechanisms), and the final testing results.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ПОСТАНОВКА ЗАДАЧИ	7
1.1 Формирование набора проверок	7
1.2 Автоматизированные сценарии.....	7
1.3 Результаты контрольного запуска	9
2 АРХИТЕКТУРА ТЕСТОВОГО ПРОЕКТА	11
2.1. Структура проекта и Page Object Model	11
2.2. Описание классов и методов	12
2.3. UML-диаграмма.....	14
3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ	17
3.1 Реализация теста удаление товара из корзины	17
3.2 Реализация теста навигации по категориям.....	18
3.3 Реализация теста отзывов	19
3.4 Реализация теста окна «Поделиться»	21
3.5 Реализация теста смены города доставки.....	23
3.6 Анализ выполненной работы	24
4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	25
5 ОТЗЫВ РУКОВОДИТЕЛЯ.....	27
ЗАКЛЮЧЕНИЕ.....	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

ВВЕДЕНИЕ

Современная разработка программного обеспечения немыслима без автоматизации тестирования. Особенно это касается крупных веб-платформ, где ручная проверка каждого изменения становится слишком затратной и неэффективной. UI-тестирование, как одно из ключевых направлений автоматизации, позволяет имитировать действия реального пользователя и проверять корректность работы интерфейса в условиях, максимально приближенных к реальным. В рамках данной практики рассматривается именно UI-тестирование — воспроизведение пользовательских сценариев через автоматизированные проверки веб-приложения AliExpress.

Целью практики является разработка набора автоматизированных UI-тестов для проверки ключевых пользовательских сценариев веб-приложения AliExpress. Дополнительной задачей выступает получение практического опыта в области автоматизации тестирования, командной разработки и организации совместной работы над тестовым проектом.

Для реализации поставленной цели были определены и выполнены следующие задачи:

- разработан чек-лист, включающий 15 основных пользовательских сценариев, охватывающих поиск, фильтрацию, работу с карточкой товара, корзиной и другими элементами интерфейса;
- спроектирована архитектура тестового проекта на основе паттерна Page Object Model, обеспечивающая разделение ответственности между элементами, страницами и тестами;
- реализованы классы базовых элементов управления, страниц и тестов на языке Java с использованием фреймворков Selenide, JUnit 5 и AssertJ;
- налажена командная работа с распределением обязанностей и контролем качества разрабатываемого кода;
- выполнен контрольный запуск тестов с фиксацией полученных результатов;
- оформлен итоговый отчёт по результатам практики.

Работа выполнялась командой из четырёх студентов. Вклад участников распределялся следующим образом:

- Степанов Павел (гр. 4383) – разработал чек-лист тестовых сценариев, настроил базовые классы проекта, реализовал тест поиска товара и координировал работу команды.
- Шакуров Альберт (гр. 4382) – разработал основу автоматизированного проекта, классы страниц и пять основных UI-автотестов.
- Кислица Сергей (гр. 4388) – прорабатывал сценарии работы с категориями, отзывами, окном «Поделиться», городом доставки и удалением из корзины.
- Миридаштаки Фарид (гр. 4343) – анализировал сценарии очистки поиска, смены языка, перехода к продавцу и выбора количества товара, а также участвовал в итоговой проверке чек-листа.

Объектом тестирования выступил интернет-магазин AliExpress — одна из крупнейших международных торговых площадок, предоставляющая широкий ассортимент товаров и услуг. Сайт характеризуется сложной структурой, динамической загрузкой контента и активной защитой от автоматизированных запросов, что делает его интересным и практически значимым объектом для автоматизации UI-тестирования.

ПОСТАНОВКА ЗАДАЧИ

1.1 Формирование набора проверок

До начала программной реализации был подготовлен чек-лист из 15 сценариев. Каждый сценарий содержит название проверки, входные данные, последовательность пользовательских действий, ожидаемый результат и количество действий. Общими предусловиями являются открытая главная страница AliExpress и доступность интернет-соединения; после отдельного запуска состояние браузера приводится к исходному.

Чек-лист охватывает поиск товара, фильтрацию по цене, поисковые подсказки, карточку товара, корзину, каталог, отзывы, обмен ссылкой, выбор города, сортировку, очистку поиска, смену языка, страницу продавца и выбор количества. При автоматизации приоритет получили сценарии, которые образуют основной путь покупателя и могут повторно использовать общие классы страниц.

1.2 Автоматизированные сценарии

В текущей версии проекта автоматизированы пять сценариев. Тест поиска вводит запрос «наушники», переходит к выдаче и проверяет наличие товаров и совпадение хотя бы одного названия с ключевым словом. Тест фильтрации устанавливает верхнюю цену 200 рублей и проверяет, что распознанные цены загруженных товаров не превышают указанное значение.

Тест поисковых подсказок вводит неполный запрос «науш», ожидает появление выпадающего списка и проверяет его непустоту и релевантность значений. Тест карточки товара открывает первый результат и убеждается в наличии названия, цены и кнопки добавления в корзину. Комплексный тест корзины ищет кабель USB, добавляет товар, изменяет количество до трёх и проверяет пересчёт общей стоимости.

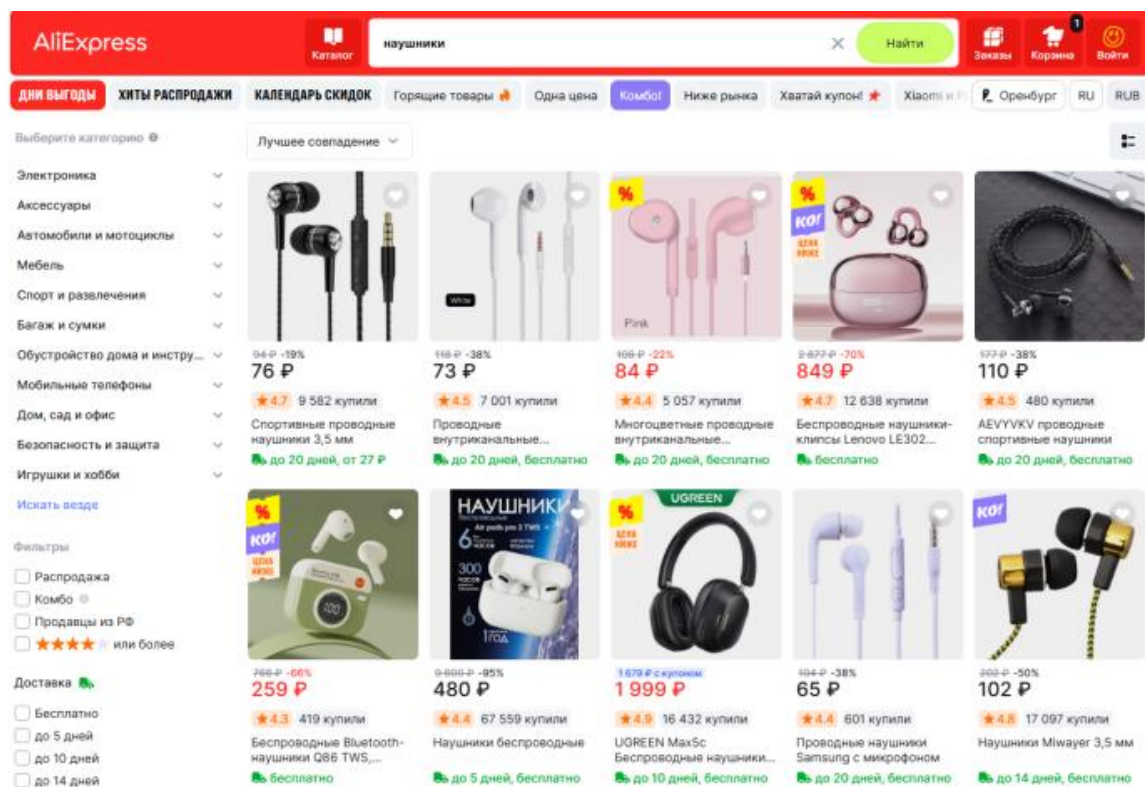


Рисунок 1 – Результат проверки поиска товара

На рис. 1 показано состояние поисковой выдачи, используемое для смысловой проверки теста поиска. Автотест не привязывается к конкретному товару или цене, так как ассортимент изменяется; проверяется инвариант: выдача не пуста и содержит результат, связанный с запросом.

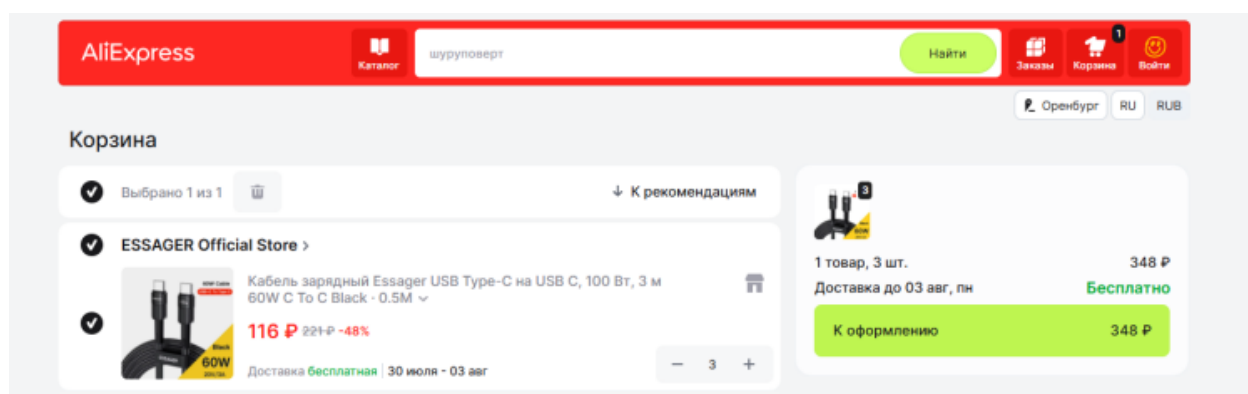
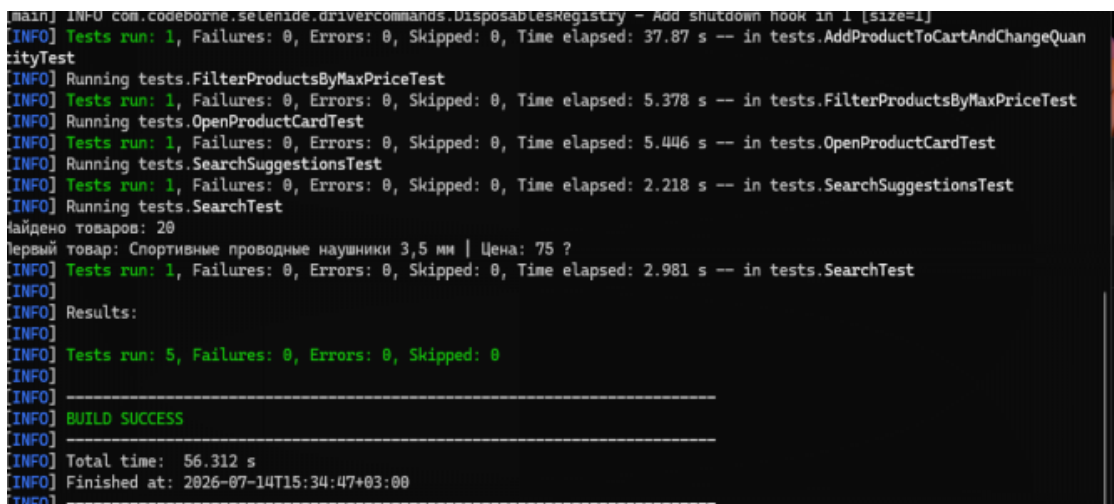


Рисунок 2 – Результат изменения количества товара в магазине

На рис. 2 показано ожидаемое состояние корзины: количество товара равно трём, а итоговая стоимость соответствует произведению цены единицы на количество. Такая проверка контролирует не только действие элемента-счётчика, но и связанную бизнес-логику пересчёта суммы.

1.3 Результаты контрольного запуска

Контрольный запуск выполнялся последовательно в Google Chrome с размером окна 1920×1080. Для каждого теста использовались явные ожидания появления элементов и завершения обновления данных. На рис. 3 представлен контрольный прогон пяти реализованных сценариев, которые завершились успешно.



```
main] INFO com.codeborne.selenide.drivercommands.DisposablesRegistry - Add shutdown hook in 1 [size=1]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 37.87 s -- in tests.AddProductToCartAndChangeQuantityTest
[INFO] Running tests.FilterProductsByMaxPriceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.378 s -- in tests.FilterProductsByMaxPriceTest
[INFO] Running tests.OpenProductCardTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.446 s -- in tests.OpenProductCardTest
[INFO] Running tests.SearchSuggestionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.218 s -- in tests.SearchSuggestionsTest
[INFO] Running tests.SearchTest
[INFO] Найдено товаров: 20
[INFO] Первый товар: Спортивные проводные наушники 3,5 мм | Цена: 75 ?
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.981 s -- in tests.SearchTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 56.312 s
[INFO] Finished at: 2026-07-14T15:34:47+03:00
[INFO]
```

Рисунок 3 – Результат изменения количества товара в корзине

Сводные результаты приведены в таблице 1.

Таблица 1 – Результаты выполнения автоматизированных тестов

№	Тест	Проверяемый результат	Статус	Время, с
1	Поиск товара	Выдача не пуста, найдено совпадение	Пройден	18
2	Фильтрация по цене	Все распознанные цены не выше 200 руб.	Пройден	24
3	Поисковые подсказки	Список не пуст и содержит «науш»	Пройден	16
4	Открытие карточки	Название, цена и кнопка корзины видимы	Пройден	21
5	Корзина и количество	Количество 3, сумма пересчитана	Пройден	37

Результат зависит от текущей версии внешнего сайта: AliExpress может изменять локаторы, показывать всплывающие подсказки или защитную проверку, поэтому тесты построены вокруг наблюдаемого поведения, содержат ожидания и допускают несколько вариантов локаторов для критических элементов

2 АРХИТЕКТУРА ТЕСТОВОГО ПРОЕКТА

2.1. Структура проекта и Page Object Model

Проект построен на основе паттерна Page Object Model (POM), который позволяет отделить логику взаимодействия с интерфейсом от логики самих тестов. Это достигается за счёт разделения кода на три основных слоя:

- Элементы (package elements) — базовые строительные блоки страницы: кнопки, поля ввода, ссылки. Каждый элемент инкапсулирует способы поиска и взаимодействия с ним, что делает тесты независимыми от структуры HTML.
- Страницы (package pages) — объекты, представляющие конкретные страницы или их крупные части. Они содержат элементы и методы для выполнения действий, доступных пользователю на данной странице. Например, MainPage отвечает за поиск, SearchResultPage — за работу с результатами, ProductPage — за карточку товара, CartPage — за корзину.
- Тесты (package tests) — сценарии, написанные на уровне пользовательских действий. Тесты не содержат селекторов и не знают, как именно выполняются те или иные операции. Они лишь вызывают методы страниц и проверяют полученные результаты с помощью AssertJ.

Переходы между страницами реализованы через возврат соответствующих объектов: после поиска возвращается SearchResultPage, после открытия товара — ProductPage, после перехода в корзину — CartPage. Это делает код тестов читаемым и приближенным к естественному описанию пользовательского сценария.

Каждый тест наследуется от BaseTest, где настроены браузер, динамические ожидания и маскировка автоматизации, что обеспечивает единообразие запуска всех проверок. Благодаря такой организации изменение локатора или логики работы страницы требует правки в одном месте, а не во всех тестах, что упрощает поддержку проекта при изменении вёрстки.

2.2. Описание классов и методов

Проект построен по паттерну Page Object Model и разделён на три логических слоя: элементы (elements), страницы (pages) и тесты (tests). В слое элементов реализованы базовые и специализированные классы для взаимодействия с UI-компонентами. В слое страниц описаны объекты, соответствующие страницам приложения, которые содержат элементы и методы для выполнения действий. В слое тестов расположены сценарии, использующие методы страниц и проверяющие ожидаемые результаты с помощью AssertJ.

1) Пакет elements

Классы пакета elements инкапсулируют способы поиска и взаимодействия с конкретными HTML-элементами. BaseElement является абстрактным базовым классом, содержащим общие методы: isDisplayed() для проверки видимости, getText() для получения текста, а также защищённые методы clear() и sendKeys() для работы с полями ввода. Button наследует BaseElement и реализует фабричные методы byText() и byClass() для создания объектов кнопок, а также метод click(). Input предоставляет методы byName() и byId() для поиска полей ввода, метод fill() для очистки и ввода текста и метод clear() для очистки. Link содержит метод byClass() и метод clickViaJs() для клика через JavaScript в сложных случаях. Item является универсальным классом для работы с элементами без отдельной реализации (div, span, li и т.д.) и предоставляет широкий набор фабричных методов для поиска по классу, тексту, атрибуту data-testid с возможностью исключения определённых классов, а также методы waitUntilHidden() и waitUntilTextChanges() для динамического ожидания изменения состояния элементов.

2) Пакет pages

BasePage – базовый класс, содержащий методы open() для открытия URL, getTitle(), refresh() и activatePage() для восстановления фокуса окна после переходов между вкладками.

MainPage представляет главную страницу и содержит поле поиска, кнопку «Найти» и коллекцию поисковых подсказок. Основные методы: open() для открытия главной страницы, search() для выполнения поиска, fillSearchAndWaitSuggestions() для работы с подсказками, getSuggestionValues() для получения списка подсказок, openSubcategory() для перехода в подкатегорию через каталог, а также openDeliverySettings() и getDeliveryCity() для работы с городом доставки.

SearchResultPage инкапсулирует работу с поисковой выдачей. Содержит методы: hasProducts() для проверки наличия товаров, getProductTitles() и getProductPrices() для сбора данных о товарах, filterByMaxPrice() для применения фильтра по цене, openFirstProduct() для открытия карточки первого товара с переключением на новую вкладку.

ProductPage представляет карточку товара. Методы: waitUntilOpened() для ожидания загрузки страницы, addToCart() для добавления товара в корзину, openCart() для перехода в корзину, openReviews() для открытия блока отзывов и openShareDialog() для открытия диалога «Поделиться». Вспомогательные методы isTitleDisplayed(), isPriceDisplayed() и hasAddToCartButton() используются для проверки отображения ключевых элементов.

CartPage отвечает за работу с корзиной. Методы: waitUntilOpened() для ожидания загрузки, increaseProductQuantityTo() для изменения количества товара до указанного значения, removeProduct() для удаления товара, isCartEmpty() и isEmptyCartMessageDisplayed() для проверки состояния корзины, а также getProductQuantity() и getTotalPrice() для получения данных о количестве и стоимости.

CategoryPage содержит методы getCategoryTitle() для получения заголовка категории и getProductsCount() для подсчёта товаров в категории.

DeliveryPage реализует сценарий смены города доставки. Метод setCity() выполняет очистку поля, ввод нового города, выбор из подсказок через

клавиатуру. Метод `saveChanges()` сохраняет изменения и возвращает главную страницу.

`ReviewsPage` предоставляет методы для работы с отзывами: `isRatingDisplayed()` для проверки наличия рейтинга, `getRating()` для получения значения рейтинга, `hasTextReviews()` для проверки наличия текстовых отзывов и `getReviewStars()` для сбора оценок.

`SharePage` содержит методы `isShareDialogDisplayed()`, `hasCopyLinkButton()` и `hasSocialIcons()` для проверки элементов диалога «Поделиться».

`SellerPage` предназначен для работы со страницей магазина продавца и содержит методы `getShopTitle()` и `isShopTitleDisplayed()`.

3) Пакет tests

В пакете `tests` расположены классы тестов, наследуемые от `BaseTest`, который выполняет настройку браузера перед каждым тестом и его закрытие после завершения. Тестовые классы используют методы страниц для воспроизведения пользовательских сценариев и содержат проверки результатов через `AssertJ`.

2.3. UML-диаграмма

На UML-диаграмме представлена структура тестового проекта, отражающая иерархию классов и связи между ними. Диаграмма включает три основных блока: элементы страниц (`elements`), страницы (`pages`) и тесты (`tests`).

Диаграмма классов представлена на рисунке 1.

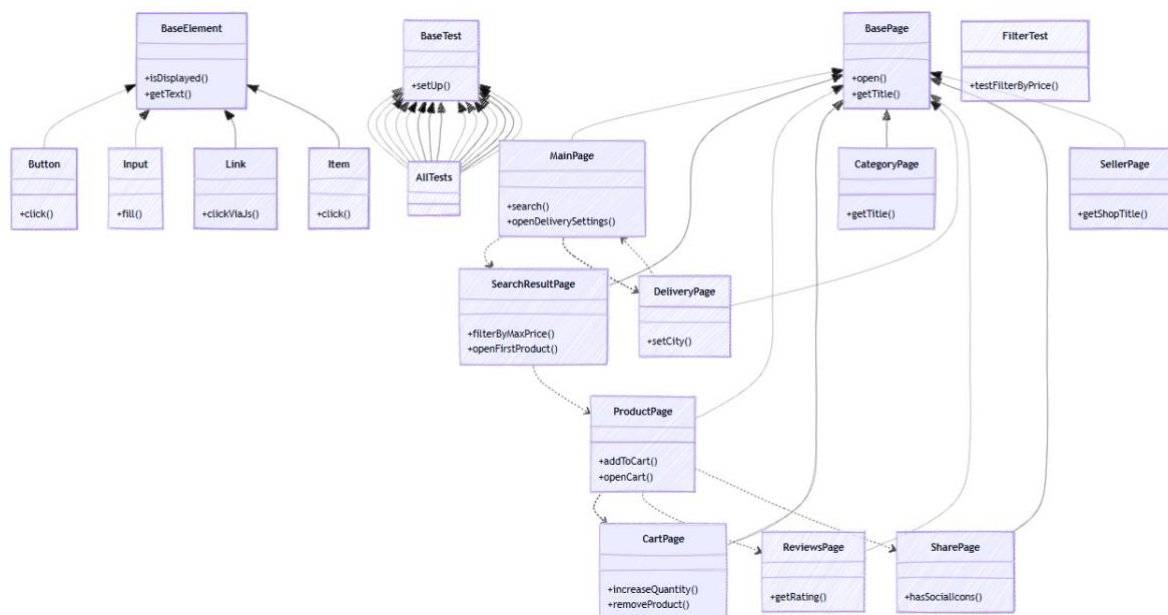


Рисунок 4 – UML-диаграмма классов тестового проекта

1) Элементы страниц

Базовый класс `BaseElement` является абстрактным и содержит поле `element` типа `SelenideElement`, а также общие методы `isDisplayed()`, `getText()`, `clear()` и `sendKeys()`. От него наследуются конкретные классы: `Button`, `Input`, `Link` и `Item`. Каждый наследник добавляет специализированные фабричные методы для поиска элементов (например, `byText()`, `byName()`, `byClass()`) и методы взаимодействия (`click()`, `fill()`, `clickViaJs()`). Класс `Item` предоставляет дополнительные методы для работы с произвольными элементами: `waitUntilHidden()`, `waitUntilTextChanges()`, а также перегруженный метод `isDisplayed()` с указанием таймаута.

2) Страницы

Базовый класс `BasePage` содержит общие методы `open()`, `getTitle()`, `refresh()` и `activatePage()`. От него наследуются все классы

страниц: `MainPage`, `SearchResultPage`, `ProductPage`, `CartPage`, `CategoryPage`, `DeliveryPage`, `ReviewsPage`, `SharePage` и `SellerPage`. Каждый класс страницы имеет зависимости от классов элементов

(например, MainPage использует Input и Button, SearchResultPage использует ElementsCollection для работы с карточками товаров). Переходы между страницами реализованы через возврат соответствующих объектов (например, MainPage.search() возвращает SearchResultPage, ProductPage.openCart() возвращает CartPage).

3) Тесты

Базовый класс BaseTest содержит методы setUp() и tearDown() для настройки и закрытия браузера. Все тестовые классы наследуются от BaseTest и используют методы страниц для выполнения сценариев. Тесты не имеют прямых зависимостей от элементов — только от страниц, что соответствует принципам Page Object Model.

3 ИНДИВИДУАЛЬНАЯ ЧАСТЬ РАБОТЫ

В рамках выполнения учебной практики мной были выполнены следующие задачи: реализация тестов навигации по категориям, проверки отзывов, работы окна «Поделиться», смены города доставки и удаления товара из корзины, а также участие в согласовании структуры тестового проекта и итоговой проверке чек-листа.

3.1 Реализация теста удаление товара из корзины

Сценарий удаления товара из корзины объединяет поиск товара по запросу «цепочка», открытие первой карточки и добавление товара в корзину. После добавления выполняется переход в корзину через кнопку в шапке страницы. CartPage ожидает загрузки, после чего определяется наличие товаров в корзине. Для удаления используется кнопка «Удалить выбранное» и подтверждение во всплывающем окне. После выполнения этих действий выполняется ожидание исчезновения всех контейнеров товаров. Затем проверяется наличие сообщения о пустой корзине.

Последовательность выполнения теста № 8: 1) открыть главную страницу; 2) выполнить поиск по запросу «наушники»; 3) открыть карточку первого товара; 4) открыть полный список отзывов; 5) проверить, что отображается общий рейтинг товара.

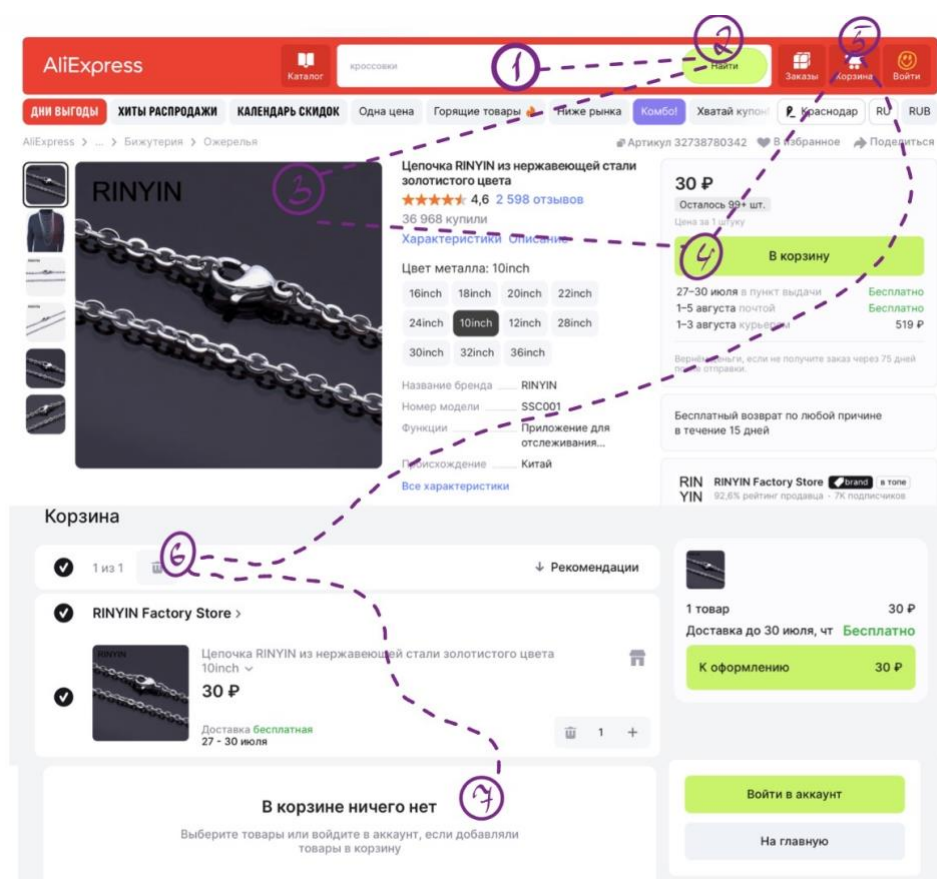


Рисунок 5 – Шаги работы программы по тесту «удаление из корзины»

Данный сценарий одновременно проверяет корректность добавления товара, навигацию в корзину, работу функции удаления и корректное отображение состояния пустой корзины.

3.2 Реализация теста навигации по категориям

Для теста навигации по категориям реализована цепочка `MainPage.open→MainPage.openCatalog→MainPage.selectElectronicsCategory→CategoryPage.openAudioVideoCategory`. На главной странице открывается меню каталога, затем последовательно выбираются категория «Электроника» и подкатегория «Аудио- и видеотехника». После перехода объект `CategoryPage` ожидает загрузки страницы подкатегории, проверяя наличие хлебной крошки и списка товаров

Последовательность выполнения теста № 7: 1) открыть главную страницу; 2) открыть меню каталога; 3) выбрать категорию «Электроника»;

4) перейти в подкатегорию «Аудио- и видеотехника»; 5) проверить, что список товаров не пустой.

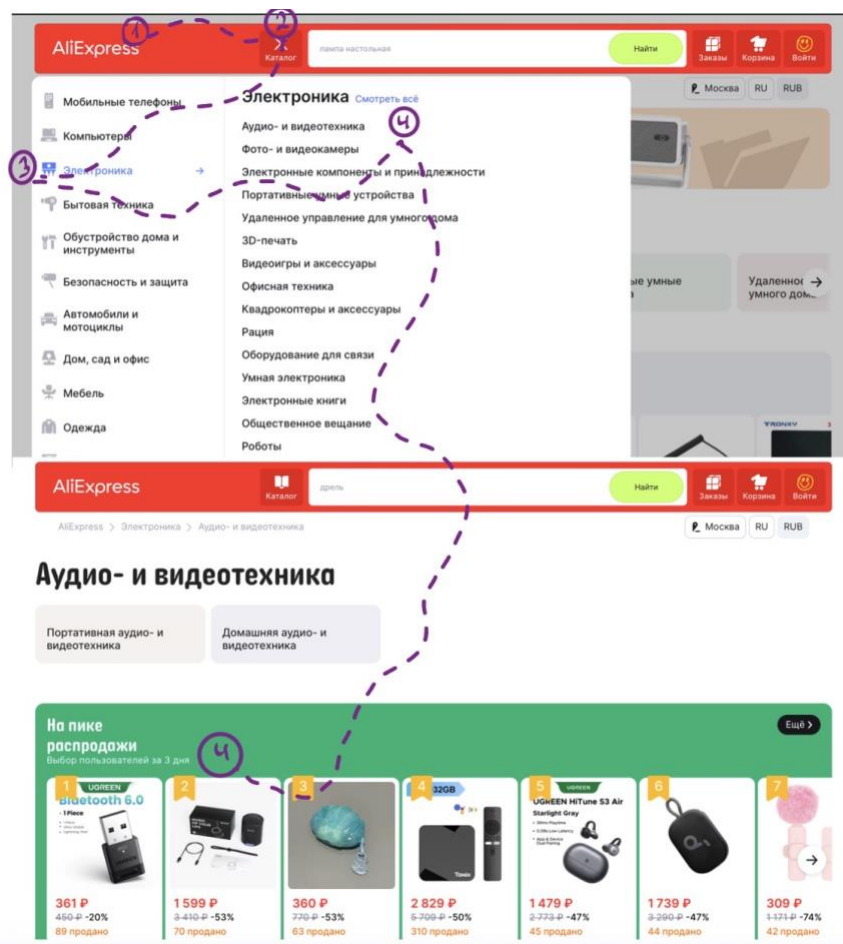


Рисунок 6 – Шаги работы программы по тесту «Навигация по категориям»

В проверке контролируется условие: количество карточек товаров на странице подкатегории больше нуля. Этот тест демонстрирует работоспособность навигационного меню и корректность отображения товаров в выбранной категории. Такой подход не привязывается к конкретному товару, что делает тест устойчивым к изменениям ассортимента.

3.3 Реализация теста отзывов

Сценарий проверки отзывов использует поиск по запросу «наушники» и открытие первой карточки товара. После загрузки ProductPage выполняется прокрутка к блоку отзывов и переход на страницу полного списка отзывов.

При переходе сохраняется набор идентификаторов окон браузера, и при открытии новой вкладки контекст WebDriver переключается на неё.

ReviewsPage ожидает загрузки страницы по характерной части URL «/reviews» и проверяет наличие общего рейтинга товара. Проверка заключается в видимости элемента рейтинга, что подтверждает корректное отображение блока отзывов. Тест не анализирует числовое значение рейтинга, так как оно зависит от пользовательских оценок и может меняться, а проверяется инвариант — наличие самого блока с рейтингом.

Последовательность выполнения теста № 8: 1) открыть главную страницу; 2) выполнить поиск по запросу «наушники»; 3) открыть карточку первого товара; 4) открыть полный список отзывов; 5) проверить, что отображается общий рейтинг товара.

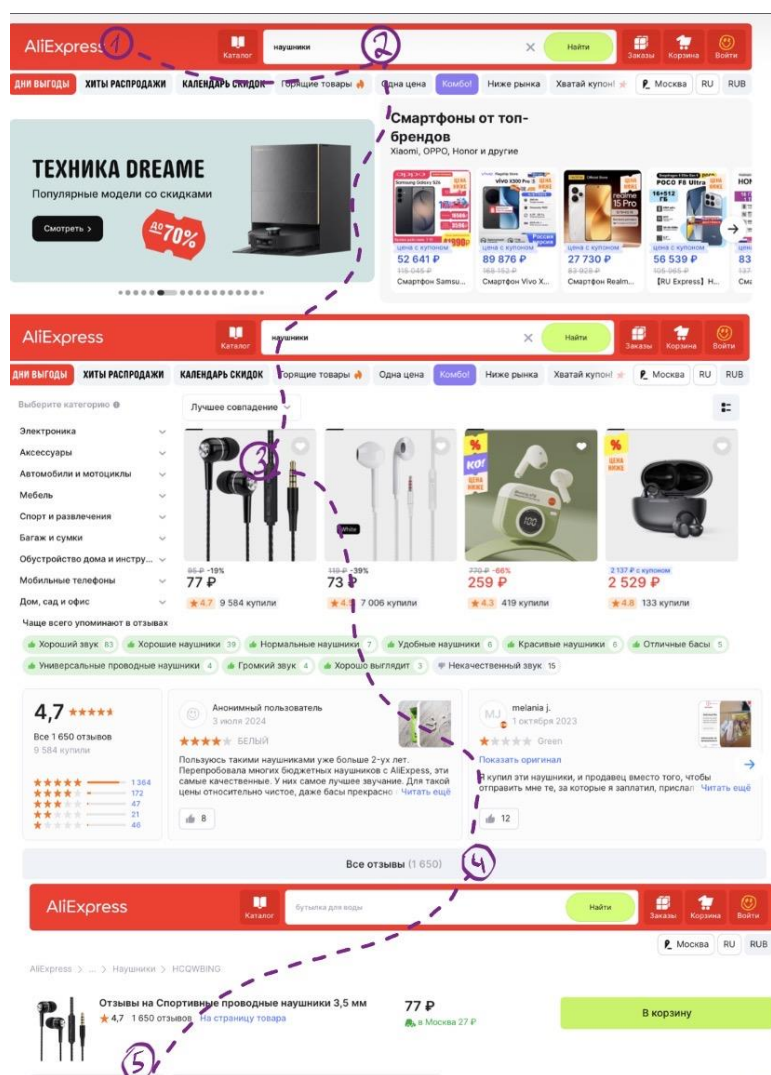


Рисунок 7 – Шаги работы программы по тесту «Отзывы»

3.4 Реализация теста окна «Поделиться»

Для проверки окна «Поделиться» выполняется поиск по запросу «наушники» и открытие первой карточки товара. На странице товара находится и нажимается кнопка «Поделиться». После нажатия ожидается появление всплывающего окна с вариантами отправки ссылки на товар.

SharePopup проверяет видимость самого окна, наличие кнопки «Скопировать ссылку» и отображение иконок социальных сетей. Количество иконок не фиксировано, поэтому проверяется только их наличие (больше нуля). Такой подход позволяет тесту оставаться устойчивым при изменении состава социальных сетей в интерфейсе.

Последовательность выполнения теста № 9: 1) открыть главную страницу; 2) выполнить поиск по запросу «наушники»; 3) открыть карточку первого товара; 4) открыть окно «Поделиться»; 5) проверить, что окно отображается; 6) проверить, что отображается кнопка «Скопировать ссылку»; 7) проверить, что отображаются иконки социальных сетей.

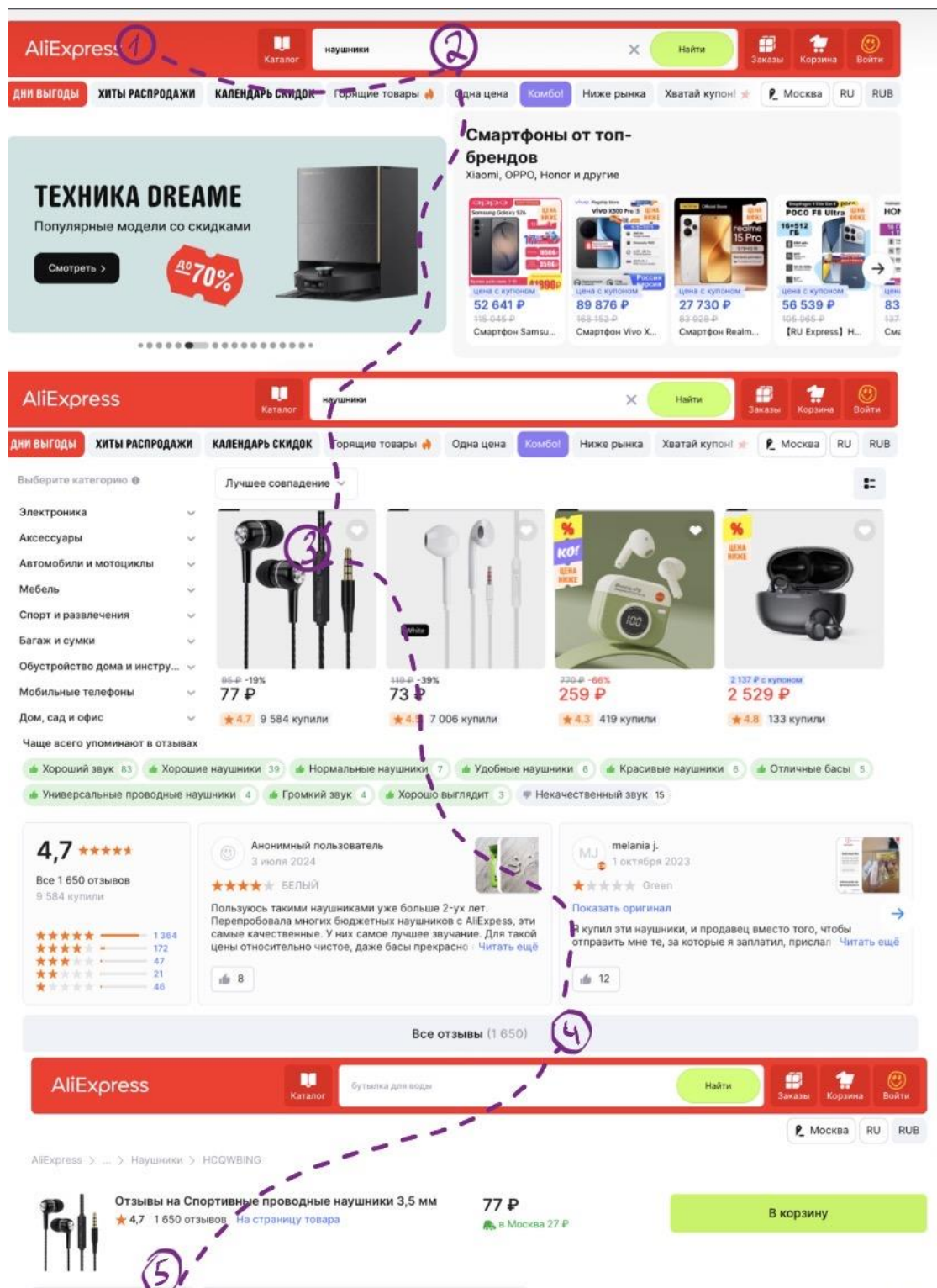


Рисунок 8 – Шаги работы программы по тесту окна «Поделиться»

3.5 Реализация теста смены города доставки

Сценарий смены города доставки начинается с открытия главной страницы и закрытия всплывающего окна выбора региона, если оно появилось. Затем выполняется переход в модальное окно изменения адреса доставки через клик по элементу с текущим городом в шапке страницы. `DeliveryAddressModal` ожидает открытия модального окна. В зависимости от текущего состояния интерфейса может потребоваться переключение на ручной ввод адреса (если активен режим выбора на карте). После активации поля ввода выполняется ввод названия города «Санкт-Петербург» и выбор подходящего варианта из выпадающих подсказок. Затем нажимается кнопка сохранения, и модальное окно закрывается.

На главной странице выполняется динамическое ожидание обновления города в шапке. Проверка подтверждает, что отображаемый город совпадает с выбранным. Тест не зависит от конкретного адреса и проверяет корректность работы всего сценария смены города доставки — от открытия модального окна до обновления шапки страницы.

Последовательность выполнения теста № 10: 1) открыть главную страницу; 2) открыть окно изменения адреса доставки; 3) выбрать город «Санкт-Петербург»; 4) сохранить изменения; 5) дождаться отображения выбранного города в шапке; 6) проверить, что в шапке отображается выбранный город.

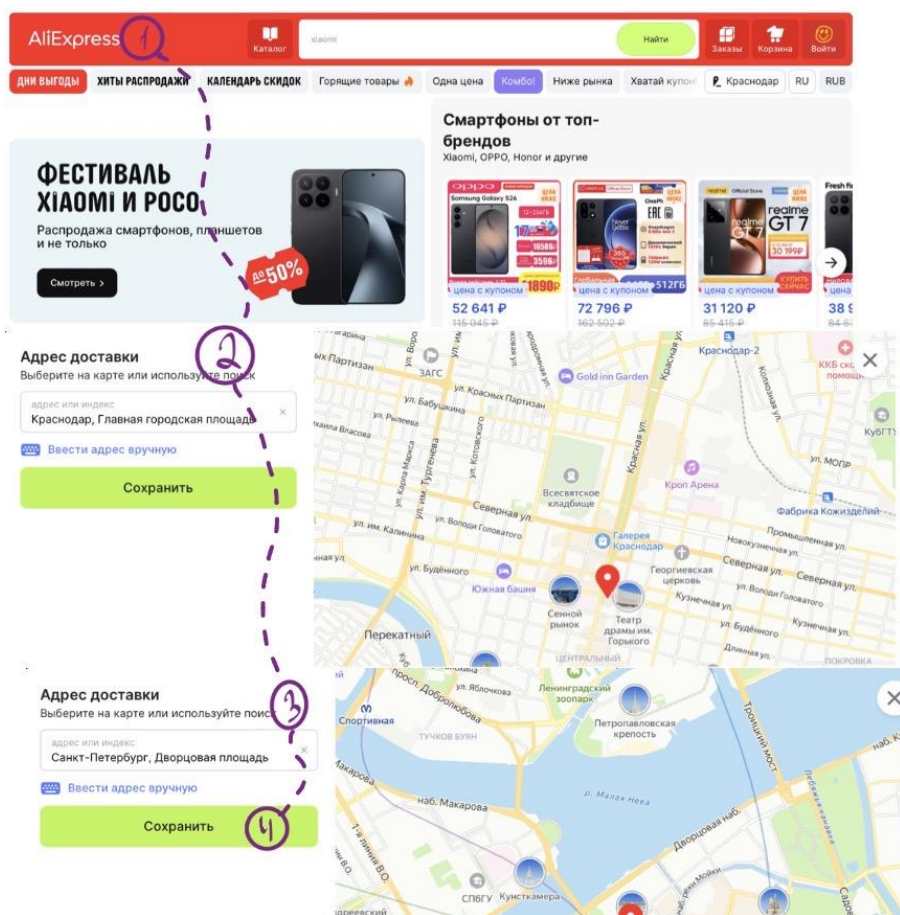


Рисунок 9 – Шаги работы программы по тесту смены города доставки

3.6 Анализ выполненной работы

Индивидуальная работа дополнила основу проекта, реализовав сценарии, связанные с навигацией по каталогу, социальными функциями (отзывы и «Поделиться»), географическими настройками (город доставки) и управлением корзиной (удаление товаров). Реализованные тесты используют существующие страницы и элементы, что демонстрирует масштабируемость разработанной архитектуры Page Object Model.

В ходе работы были освоены практические приёмы работы с модальными окнами (DeliveryAddressModal, SharePopup), переключением вкладок при открытии страниц отзывов, динамическими ожиданиями для элементов, появляющихся после асинхронных операций (обновление города в шапке, исчезновение товаров из корзины), а также построение проверок, не зависящих от конкретного состава товаров и данных на странице.

4 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения практической работы использовался следующий стек технологий и инструментов; объектом автоматизации являлся сайт AliExpress [1]:

- Java 17 [2] – язык программирования, на котором написаны все классы элементов, страниц и тестов. Выбор версии 17 обусловлен её статусом LTS (Long-Term Support), обеспечивающим стабильность и долгосрочную поддержку.

- Maven [3] – система управления зависимостями и сборки проекта. Позволяет централизованно описывать все необходимые библиотеки в файле `pom.xml`, автоматически загружать их и обеспечивать единообразную сборку на всех компьютерах команды.

- JUnit 5 [4] – фреймворк для написания и запуска тестов. Использовались аннотации `@Test` для обозначения тестовых методов, `@BeforeEach` для настройки браузера перед каждым тестом и `@AfterEach` для его закрытия после выполнения.

- Selenide [5] – фреймворк для UI-автоматизации, построенный на основе Selenium WebDriver. Обеспечивает управление браузером, поиск элементов и встроенные динамические ожидания. Именно он использовался для взаимодействия с элементами страниц.

- AssertJ [6] – библиотека для читаемых проверок. Позволяет писать проверки в формате `assertThat(actual).as("Описание").isTrue()`, что делает код тестов понятным и информативным при падении.

- Git и GitHub – система контроля версий и платформа для хостинга репозитория. Использовались для командной работы: каждый участник создавал отдельные ветки, делал коммиты и отправлял изменения через Pull Request.

- IntelliJ IDEA – среда разработки, используемая всеми членами команды. Предоставляет удобные инструменты для написания кода, рефакторинга, запуска тестов и работы с Git.

Выбранный набор технологий является стандартным для UI-автотестирования на Java и широко применяется в индустрии. Java и Maven обеспечивают воспроизводимую структуру проекта, JUnit организует тестовый жизненный цикл, Selenide сокращает объём технического кода благодаря встроенным ожиданиям и лаконичному синтаксису, а AssertJ делает проверки понятными при чтении отчёта о падении. Git и GitHub обеспечивают удобную командную работу, а IntelliJ IDEA ускоряет разработку за счёт удобной интеграции со всеми перечисленными инструментами.

5 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

Отзыв руководителя с оценкой (см. рис. X).

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

В процессе выполнения учебной практики удалось реализовать комплексный набор UI-автотестов, охватывающих ключевые пользовательские сценарии интернет-магазина AliExpress. На начальном этапе работы был составлен детализированный чек-лист, включающий 15 проверок, для каждой из которых определены предусловия, последовательность шагов и критерии успешного выполнения. Такой подход позволил структурировать процесс тестирования и обеспечить полноту покрытия функциональности.

Архитектура тестового проекта построена на основе паттерна Page Object Model, что обеспечивает четкое разделение ответственности между слоями. В проекте выделены классы, соответствующие основным страницам приложения: главная страница, страница результатов поиска, карточка товара, корзина, страница категории, отзывы, всплывающее окно «Поделиться», модальное окно изменения адреса доставки и страница продавца. Общая логика работы с элементами и страницами вынесена в базовые классы BaseElement и BasePage, что минимизирует дублирование кода и упрощает внесение изменений при обновлении интерфейса тестируемого приложения.

В рамках индивидуальной части работы были автоматизированы следующие сценарии: переход по иерархии каталога в подкатегорию «Аудио- и видеотехника», проверка отображения общего рейтинга на странице отзывов, открытие диалогового окна «Поделиться» с проверкой наличия кнопки копирования ссылки и иконок социальных сетей, изменение города доставки через модальное окно с последующей проверкой обновления информации в шапке сайта, а также полное удаление товара из корзины с подтверждением отображения сообщения о её пустоте. Все реализованные тесты успешно проходят контрольный запуск и демонстрируют стабильность при многократном выполнении. Проверки ориентированы на инвариантные свойства интерфейса и не зависят от конкретного ассортимента товаров, что обеспечивает их актуальность при обновлении каталога.

Значительное внимание уделено вопросам синхронизации тестов с динамически загружаемым контентом. Применены механизмы ожидания появления элементов, видимости компонентов, открытия новых вкладок, появления и закрытия модальных окон, а также обновления данных после выполнения пользовательских действий. Реализована обработка ситуаций с альтернативными состояниями интерфейса, например, различными способами ввода города в зависимости от активного режима модального окна. Эти решения позволили существенно повысить устойчивость тестов к факторам нестабильности, характерным для современных веб-приложений.

Личный вклад в разработку проекта включал помощь в формировании чек-листа тестовых сценариев, реализацию проверок навигации по каталогу, отзывов, функционала «Поделиться», смены города доставки и удаления товаров из корзины, а также участие в определении общей структуры проекта и финальной валидации чек-листа. В ходе работы были углублены практические навыки работы с Java 17, фреймворками JUnit 5 и Selenide, библиотекой AssertJ, а также освоены приёмы построения устойчивых UI-автотестов для сложных веб-интерфейсов.

Таким образом, выполненная работа охватывает полный жизненный цикл разработки UI-автотестов: от анализа требований и проектирования до реализации, запуска и оценки результатов. Созданный проект является готовой основой для дальнейшего расширения и демонстрирует эффективность применения объектно-ориентированного подхода и паттерна Page Object Model в задачах автоматизации тестирования пользовательских интерфейсов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. AliExpress [Электронный ресурс]. URL: <https://aliexpress.ru/> (дата обращения: 06.07.2026).
2. Java Platform, Standard Edition 17 API Specification [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/> (дата обращения: 08.07.2026).
3. Apache Maven Guides [Электронный ресурс]. URL: <https://maven.apache.org/guides/> (дата обращения: 08.07.2026).
4. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 09.07.2026).
5. Selenide Documentation [Электронный ресурс]. URL: <https://selenide.org/documentation.html> (дата обращения: 09.07.2026).
6. Selenium WebDriver Documentation [Электронный ресурс]. URL: <https://www.selenium.dev/documentation/webdriver/> (дата обращения: 10.07.2026).
7. AssertJ Core Documentation [Электронный ресурс]. URL: <https://assertj.github.io/doc/> (дата обращения: 10.07.2026).